SEMINAR · PRÜFUNG GH-300

# GitHub Copilot

## Vom Vorschlag zum Agenten: Arbeiten, Anpassen, Verwalten

Dr.-Ing. Grigory Devadze 23.10.2026

GH-300

# Was Sie mitnehmen

- Copilot in VS Code **gezielt einsetzen:** Inline-Vorschläge, Chat, Ask, Plan und Agent, und den **Kontext** steuern
- Aufträge so formulieren, dass das Ergebnis **prüfbar** ist: Anforderung, Plan, enger Änderungsauftrag, Debug-Prompt
- Mit Copilot **testen und Fehler suchen**, ohne dass die Tests ihre Aussage verlieren
- Copilot für ein Repository **anpassen:** Instruktionen, Prompt Files, Custom Agents, MCP-Server
- Copilot **außerhalb der IDE** und die **Verwaltung** so weit beherrschen, wie die Prüfung GH-300 sie verlangt

# Ablauf

1. **Verantwortungsvolle KI und wie Copilot arbeitet**
2. **Copilot in der IDE und Prompt Engineering**
3. **Entwickleraufgaben:** Tests, Fehlersuche, Refactoring
4. **Copilot anpassen:** Instruktionen, Prompt Files, Agenten, MCP
5. **Copilot außerhalb der IDE:** CLI, GitHub.com, Cloud Agent
6. **Verwaltung, Datenschutz und Prüfung**

TEIL I

# Verantwortungsvolle KI und wie Copilot arbeitet

Was mit Ihrem Code passiert, bevor ein Vorschlag erscheint, und wer für das Ergebnis verantwortlich ist.

# Lernziele

## Prinzipien

Sie nennen Microsofts sechs Prinzipien verantwortungsvoller KI und die Grenzen generativer KI.

## Datenfluss

Sie beschreiben, welchen Kontext Copilot sammelt und welche Filter vor und nach dem Modell greifen.

## Einsatz

Sie schätzen ein, wofür Copilot taugt, wo Sie genauer prüfen und wann Code fertig ist.

## Pläne

Sie kennen die Funktionsunterschiede der Pläne, die für Kurs und Prüfung zählen.

# Sechs Prinzipien verantwortungsvoller KI

- **Fairness**: alle Menschen fair behandeln
- **Reliability and safety**: zuverlässig und sicher arbeiten
- **Privacy and security**: Daten schützen, Privatsphäre achten
- **Inclusiveness**: alle Menschen einbeziehen und befähigen
- **Transparency**: nachvollziehbar sein
- **Accountability**: Menschen bleiben für KI-Systeme verantwortlich

Quelle: Microsoft Learn, Responsible AI with GitHub Copilot

# Grenzen generativer KI

- **plausibel wirkender, falscher Code**
- Bias aus den Trainingsdaten
- Sicherheitslücken im erzeugten Code
- Treffer mit **öffentlichem Code**
- schwächer bei seltenen Sprachen und bei Fragen außerhalb von Code; am besten auf Englisch

> **Sie verantworten das Ergebnis.** Copilot schlägt vor; Sie lesen, testen und entscheiden, was übernommen wird.

Quelle: GitHub Docs, Responsible use of GitHub Copilot Chat

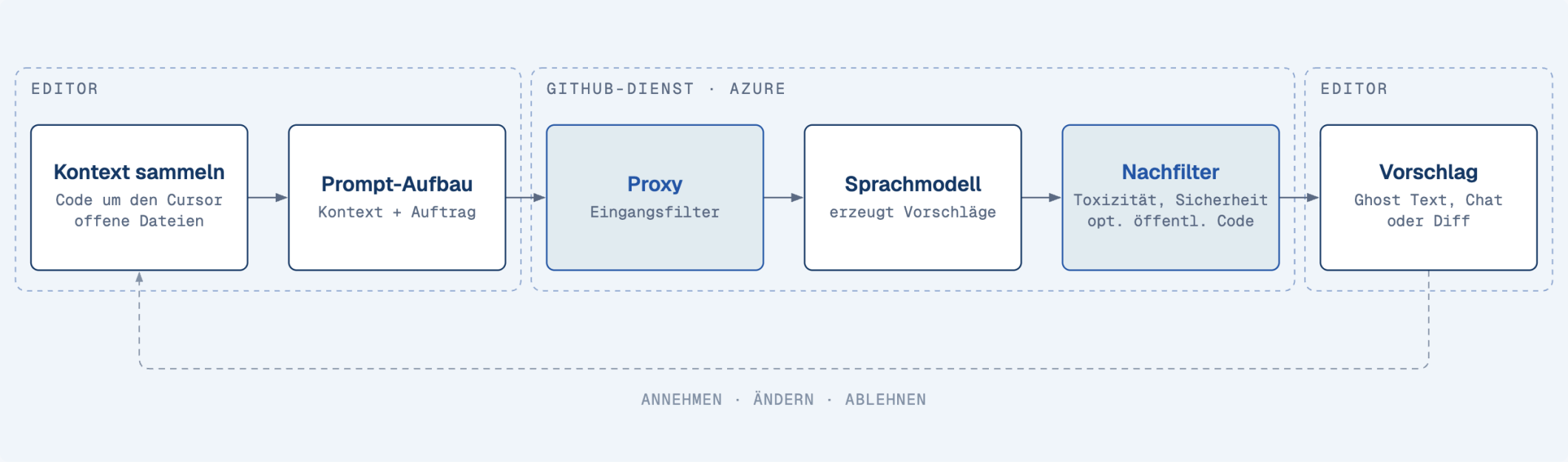
# Kontext eines Vorschlags

- Code **vor und nach dem Cursor** (Fill-in-the-Middle)
- Dateiname und Dateityp
- **benachbarte offene Tabs**
- Projektstruktur und Pfade, Sprachen und Frameworks

> Was nicht im Kontext steht, kennt Copilot nicht. Öffnen Sie verwandte Dateien; im Chat geben Sie Dateien mit #-Referenzen mit.

Quelle: Microsoft Learn, GitHub Copilot user prompt process flow; VS Code Docs, Inline suggestions

# Datenfluss eines Vorschlags




> Nur geprüfte Vorschläge erreichen den Editor. Was ein Filter beanstandet, wird gekürzt oder verworfen.

Quelle: Microsoft Learn, GitHub Copilot user prompt process flow

# Filter vor und nach dem Modell

## Vor dem Modell

- Proxy blockt **Prompt-Hacking** und Versuche, Systemdetails zu entlocken
- Toxizitätsfilter: Hassrede, unangemessene Inhalte, **personenbezogene Daten**

## Nach dem Modell

- Toxizitätsfilter auf die Ausgabe
- Prüfung auf Codequalität und Sicherheitslücken, etwa XSS oder SQL Injection
- optional **Filter für öffentlichen Code:** Abgleich mit rund 150 Zeichen Umgebung

> Die Filter ersetzen kein Review. Ein Vorschlag kann alle Filter passieren und trotzdem falsch sein.

# Stärken und Schwächen im Entwickleralltag

## Hier hilft Copilot

- **Gerüste und Klebecode:** Dateistruktur, Testvorlagen, Datenklassen
- **Erklären:** fremder Code, Fehlermeldungen, Stack Traces
- **Testideen:** Grenzfälle, an die Sie nicht sofort denken
- **Umbauten nach Muster:** umbenennen, migrieren, Duplikate auflösen

## Hier prüfen Sie genauer

- **erfundene APIs**, die plausibel klingen
- **veraltetes Wissen** über Bibliotheken und Versionen
- **begrenzter Kontext:** Copilot sieht nur, was mitgeschickt wird
- **Selbstsicherheit:** falsche Antworten klingen so überzeugt wie richtige

> Je weiter eine Aufgabe von verbreitetem Code entfernt ist, desto genauer prüfen Sie das Ergebnis.

# Definition of Done

Code gilt als fertig, wenn

1. er **läuft** und das erwartete Ergebnis liefert,
2. **Tests** Normalfall und Grenzfälle abdecken,
3. Sie den **Diff gelesen** und verstanden haben, nicht nur die Zusammenfassung,
4. er **sicher** ist: keine Geheimnisse im Code, keine unsicheren Voreinstellungen,
5. die **Commit-Nachricht** das Warum erklärt.

> „Copilot hat es gebaut, sieht gut aus“ ist keine Abnahme.

# Pläne und AI Credits

- Für Einzelne: **Free**, **Student**, **Pro**, **Pro+**, **Max**
- Für Organisationen: **Business** und **Enterprise**, abgerechnet je zugewiesenem Platz
- Seit 01.06.2026 zählen **AI Credits**: Verbrauch nach Modell und Tokens, 1 Credit = 0,01 USD
- Credits verbrauchen Chat, CLI, Cloud Agent, Code Review, Spaces und Drittanbieter-Agenten
- Das Kontingent beginnt am 1. des Monats neu, Reste verfallen

> **Vervollständigungen und Next Edit Suggestions** verbrauchen keine Credits: in bezahlten Plänen unbegrenzt, in Free 2.000 im Monat.

Quelle: GitHub Docs, Plans for GitHub Copilot; Usage-based billing for individuals

# Funktionen je Plan

| Funktion | Free | Student | Pro | Pro+ | Max | Business | Enterprise |
|---|---|---|---|---|---|---|---|
| Modellwahl | nur Auto | nur Auto | Auswahl | Premium | Vorrang Premium | Premium | Vorrang Premium |
| Agent, MCP, CLI | ja | ja | ja | ja | ja | ja | ja |
| Cloud Agent | nein | ja | ja | ja | ja | ja | ja |
| Code Review | nur Auswahl in VS Code | ja | ja | ja | ja | ja | ja |
| PR-Zusammenfassungen | nein | ja | ja | ja | ja | ja | ja |
| Drittanbieter-Agenten | nein | nein | ja | ja | ja | ja | ja |
| Audit Logs | nein | nein | nein | ja | ja | ja | ja |
| Org-Richtlinien, Content Exclusion | nein | nein | nein | nein | nein | ja | ja |

> Free hat keinen Cloud Agent. Organisationsrichtlinien und Content Exclusion gibt es nur in **Business** und **Enterprise**.

Quelle: GitHub Docs, Plans for GitHub Copilot

# Recap Teil I

- **Sechs Prinzipien**; im Alltag zählt vor allem: Sie verantworten, was Sie übernehmen
- Copilot sieht nur den **Kontext**, den Editor und Chat mitschicken
- **Filter** vor und nach dem Modell, optional gegen öffentlichen Code
- Gut bei Gerüsten, Erklärungen und Testideen; genauer prüfen bei APIs, Versionen und Sicherheit
- Die **Definition of Done** bleibt bei Ihnen
- Der **Plan** bestimmt den Funktionsumfang, abgerechnet wird in **AI Credits**

TEIL II

# Copilot in der IDE und Prompt Engineering

Vorschläge, Chat und Agenten in VS Code, und Aufträge,
deren Ergebnis Sie prüfen können.

# Lernziele

## Vorschläge steuern

Sie nehmen Inline-Vorschläge und Next Edit Suggestions an, verwerfen sie und blättern durch Alternativen.

## Agenten wählen

Sie setzen Ask, Plan und Agent passend zur Aufgabe ein und erteilen Freigaben bewusst.

## Kontext geben

Sie binden Dateien und Werkzeuge mit #-Referenzen, @-Teilnehmern und Slash-Befehlen ein.

## Prüfbar beauftragen

Sie formulieren Anforderung, Plan, Randbedingungen und enge Änderungsaufträge.

# Inline-Vorschläge und Next Edit Suggestions

- **Ghost Text:** Beim Tippen zeigt Copilot einen gedimmten Vorschlag
- **Next Edit Suggestions** (NES) sagen Ort und Inhalt der nächsten Änderung voraus; ein Pfeil im Gutter markiert die Stelle
- Beides verbraucht **keine AI Credits**

| Aktion | Windows, Linux |
|---|---|
| Vorschlag annehmen | `Tab` |
| Vorschlag verwerfen | `Esc` |
| nächster, vorheriger Vorschlag | `Alt+]`, `Alt+[` |
| Vorschlag auslösen | `Alt+\` |
| weitere Vorschläge im eigenen Bereich | `Ctrl+Enter` |
| NES: zur Stelle springen, dann übernehmen | `Tab`, `Tab` |

Quelle: GitHub Docs, Keyboard shortcuts for GitHub Copilot; VS Code Docs, Inline suggestions

# Chat-Ansichten

| Ansicht | Windows, Linux | macOS |
|---|---|---|
| **Chat-Ansicht** (Seitenleiste, Editor-Tab oder eigenes Fenster) | `Ctrl+Alt+I` | `^⌘I` |
| **Inline-Chat** im Editor | `Ctrl+I` | `⌘I` |
| **Inline-Chat** im Terminal | `Ctrl+I` | `⌘I` |
| **Quick Chat** | `Ctrl+Shift+Alt+L` | `⇧⌥⌘L` |
| neuer Chat | `Ctrl+N` | `⌘N` |

> Der Inline-Chat bezieht sich auf den aktiven Editor. Gehört die Datei schon zu einer laufenden Chat-Sitzung, öffnet `Ctrl+I` stattdessen „Ask in Chat“.

Quelle: VS Code Docs, Chat view; Inline chat and Quick Chat

# Harness und Agenten

Der **Harness** legt fest, womit eine Sitzung läuft: **Copilot** (empfohlen), **Local**, Harnesses weiterer Anbieter oder **Cloud** (liefert einen Pull Request). Der **Agent** legt fest, was sie darf.

| Agent oder Modus | Was er tut |
|---|---|
| **Ask** | beantwortet Fragen, liest und durchsucht Code, ändert keine Dateien |
| **Plan** | recherchiert und legt einen Plan zur Prüfung vor; auch mit `/plan` |
| **Agent** | setzt Änderungen um und prüft das Ergebnis: Dateien, Terminal, Tools |
| **Autopilot** | genehmigt alle Tools, wiederholt bei Fehlern, beantwortet Rückfragen selbst |
| **Edit** | abgekündigt seit VS Code 1.126; die Prüfung nennt ihn noch als „Copilot Edits“ |

Quelle: VS Code Docs, Agent harnesses; Approvals and permissions; VS Code 1.126 Release Notes

# Freigaben und Checkpoints

- Standard ist **Manual permissions:** Sie geben Werkzeuge einzeln frei, einmal, für die Sitzung, den Workspace oder immer
- Terminal: lesende Standardbefehle laufen ohne Rückfrage, `rm` und `del` fragen nach; eigene Regeln in `chat.tools.terminal.autoApprove`, eine Regel mit `false` gewinnt
- `/yolo` oder `chat.tools.global.autoApprove` geben alles frei, VS Code warnt davor
- Freigaben zurücknehmen: **Chat: Reset Tool Confirmations**
- **Checkpoints:** Vor jeder Anfrage sichert VS Code die betroffenen Dateien; **Restore Checkpoint** setzt Dateien und Verlauf zurück

> Ein Checkpoint nimmt **keine** Terminalbefehle, Netzwerkaufrufe oder Deployments zurück. Lesen Sie jeden Befehl, bevor Sie ihn freigeben.

Quelle: VS Code Docs, Approvals and permissions; Review code edits

# Kontext mit #-Referenzen

Mit `#` geben Sie Dateien und Werkzeuge gezielt mit. Seit VS Code 1.107 tragen viele Werkzeuge einen Gruppennamen; die alten Namen funktionieren weiter.

| Zweck | aktuell in VS Code | ältere Schreibweise |
|---|---|---|
| Datei, Ordner, Symbol, Auswahl | `#<datei>`, `#<ordner>`, `#<symbol>`, `#selection` | `#file`, `#sym` (GitHub Docs) |
| Codebasis semantisch durchsuchen | `#search/codebase` | `#codebase` |
| Probleme im Editor | `#read/problems` | `#problems` |
| geänderte Dateien | `#search/changes` | `#changes` |
| fehlgeschlagener Test | `#execute/testFailure` | `#testFailure` |
| Webseite abrufen | `#web/fetch` | `#fetch` |
| Auswahl im Terminal | `#read/terminalSelection` | `#terminalSelection` |

> `#codebase` durchsucht einen Index semantisch und lädt **nicht** das ganze Repository in den Kontext.

Quelle: VS Code Docs, Tools and context reference; GitHub Docs, Copilot Chat cheat sheet

# @-Teilnehmer und Slash-Befehle

## @-Teilnehmer

- `@github`
- `@terminal`
- `@vscode`, etwa `@vscode /search`
- `@azure` (Preview, nur in den GitHub Docs)
- `@workspace` fehlt in den aktuellen Doku-Seiten zum Chat; für die Codebasis gilt `#codebase`

## Slash-Befehle

- `/explain`, `/fix`, `/tests`
- `/doc` im Inline-Chat
- `/plan` startet den Agenten Plan
- `/init` erzeugt Workspace-Instruktionen
- `/fixTestFailure` (laut GitHub Docs)
- `/<name>` ruft Prompt-Dateien und Skills auf

Quelle: VS Code Docs, AI features cheat sheet; Add context; GitHub Docs, Copilot Chat cheat sheet

# Modellwahl

- Modell im **Chat-Eingabefeld** wählen oder **Auto** nehmen; bezahlte Pläne erhalten bei Auto 10 % Rabatt auf die Modellkosten
- Free und Student: nur **Auto**
- Reasoning-Modelle haben das Untermenü **Thinking Effort**; mehr Effort kostet mehr AI Credits
- In nicht vertrauenswürdigen Workspaces (Restricted Mode) zeigt der Picker nur Auto
- **BYOK**: Ein eigener API-Schlüssel ist möglich; Inline-Vorschläge und semantische Suche brauchen trotzdem den Copilot-Dienst

Quelle: VS Code Docs, Language models; GitHub Docs, Plans for GitHub Copilot

# Anforderung zuerst

Die Anforderung steht in einer Datei, bevor Copilot Code schreibt:

```
## Ziel
`pruefe_anmeldung` in `auth/sperre.py` sperrt ein Konto nach fünf Fehlversuchen.

## Akzeptanzkriterien
- Given 4 Fehlversuche, When ein weiterer Fehlversuch, Then ist das Konto gesperrt.

## Grenzfälle
- Eine erfolgreiche Anmeldung setzt den Zähler auf 0 zurück.

## Fehlerverhalten
- Unbekannter Benutzername: `KontoUnbekannt`, der Zähler bleibt unverändert.

## Nicht im Umfang
- Entsperren durch den Support, E-Mail an den Benutzer
```

- **Given/When/Then** zwingt zu prüfbaren Aussagen und wird fast direkt zum Test
- **Nicht im Umfang** hält den Auftrag klein
- Copilot liest die Datei mit, wenn Sie sie mit # referenzieren

# Plan zuerst

Agent **Plan** wählen oder `/plan` voranstellen:

```
Lies #file:anforderung.md und #file:auth/sperre.py.
Schlag einen Plan vor und schreib noch keinen Code:
- Welche Dateien und Funktionen änderst du, in welcher Reihenfolge?
- Welche Risiken und Annahmen siehst du?
Warte auf meine Freigabe.
```

- Lesen Sie den Plan gegen die Anforderung: Steht das dort, oder hat Copilot es angenommen?
- Antworten auf Rückfragen tragen Sie in die Anforderungsdatei ein
- Erst danach wechseln Sie zum Agenten **Agent**

# Randbedingungen

```
## Technik
- Python 3, nur Standardbibliothek, Tests mit pytest

## Architektur
- Fachlogik ohne Ein- und Ausgabe, kein globaler Zustand

## Sicherheit
- keine Passwörter oder Tokens in Logs und Fehlermeldungen

## Arbeitsweise
- vorhandene Tests nicht ändern, neue Abhängigkeiten nur nach Rückfrage
```

> Was für jeden Auftrag gilt, gehört in **`.github/copilot-instructions.md`**. Copilot Chat in VS Code liest diese Datei mit; `/init` legt einen ersten Entwurf an. Details in Teil IV.

Quelle: VS Code Docs, Custom instructions

# Enger Änderungsauftrag

„Schreib die Datei neu“ ändert mehr als nötig. Besser:

```
Ändere in #file:auth/sperre.py nur die Funktion `pruefe_anmeldung`:
- neuer Parameter `max_versuche: int = 5`
- ohne Argument verhält sie sich wie bisher
Ändere keine anderen Funktionen und keine Tests.
Führ danach `python -m pytest` aus und zeig mir die Ausgabe.
```

- Enger Umfang ergibt einen kleinen Diff und ein schnelles Review
- „nur“ und „keine anderen“ verhindern ungebetene Verbesserungen
- Den Diff lesen Sie trotzdem selbst, bevor Sie die Änderung übernehmen

# Erst erklären oder erst implementieren

**Erst erklären:** bei unbekanntem Problem oder offener Architektur

```
Schreib noch keinen Code.
1. Welche Ansätze kommen für <Problem> infrage?
2. Was spricht für und gegen jeden?
3. Welchen empfiehlst du, und warum?
```

**Erst implementieren:** bei Routine und klarer Anforderung

```
Setz #file:anforderung.md um.
Danach: Nenn in drei Sätzen die wichtigsten
Entscheidungen und jede Annahme, die nicht
in der Anforderung stand.
```

> Zum Erklären reicht der Agent **Ask**: Er ändert keine Dateien.

# Anti-Pattern beim Prompten

- **„Mach es besser“**: Ohne Kriterium entsteht Veränderung, keine Verbesserung
- **„Was meinst du?“**: Das Modell weicht aus oder erfindet eine Begründung
- **Mehrere Aufgaben in einem Prompt**: Das Ergebnis ist ein Kompromiss
- **Keine Randbedingungen**: Copilot trifft Annahmen, die Sie nicht teilen
- **Nur Verbote** („nicht X, nicht Y“): Sagen Sie, was Sie wollen
- **Agent für eine Frage**: Für Erklärungen reicht Ask

# Recap Teil II

- **Inline-Vorschläge** und **NES** kosten keine Credits; `Tab`, `Esc`, `Alt+]` steuern sie
- **Ask** fragt, **Plan** plant, **Agent** setzt um; Autopilot gibt alle Tools frei
- **Freigaben** bewusst erteilen; Checkpoints nehmen keine Terminalbefehle zurück
- Kontext mit `#`, `@` und `/` gezielt geben; `#codebase` sucht, lädt aber nicht alles
- **Anforderung**, **Plan**, **Randbedingungen**, **enger Auftrag**: Jedes Muster macht das Ergebnis prüfbar

TEIL III

# Entwickleraufgaben: Tests, Fehlersuche, Refactoring

Copilot schreibt Tests, sucht Fehler und baut um. Sie legen fest, was als richtig gilt.

# Lernziele

## Testen

Testfälle von Copilot vorschlagen lassen, auswählen und gezielt umsetzen

## Fehlersuche

Fehler mit Hypothesen und Gegenprüfung eingrenzen, bevor sich Code ändert

## Refactoring

Umbauten mit festen Invarianten beauftragen und das Ergebnis prüfen

## Review und Doku

Review, Docstring und Commit-Nachricht mit Copilot vorbereiten

# Teststrategie mit Copilot

Ideen und Umsetzung getrennt halten:

1. **Fälle erfragen:** im Agenten **Ask**, mit Begründung, noch ohne Testcode
2. **Auswählen:** Sie streichen, was fachlich keine Rolle spielt
3. **Umsetzen lassen:** mit `/tests` auf der markierten Funktion oder im **Agent** mit genauer Fallliste
4. **Erwartete Werte prüfen:** jeder Wert stammt aus der Anforderung

> Wer sofort `/tests` aufruft, bekommt die Fälle, die Copilot naheliegend findet. Wer vorher auswählt, bekommt die Fälle, die das Projekt braucht.

Quelle: VS Code Docs, AI features cheat sheet; VS Code Docs, Agent harnesses

# Test-Ideen erfragen

```
#file:liste/seiten.py #file:tests/test_seiten.py #file:docs/anforderung.md

Welche Fälle aus der Anforderung decken die vorhandenen Tests nicht ab?
1. Grenzwerte: 0, 1, genau eine volle Seite, ein Eintrag mehr
2. Ungültige Eingaben und die erwartete Fehlermeldung
3. Fälle, die nur in der Anforderung stehen, nicht im Code

Liefere je Fall: Eingabe, erwartetes Ergebnis laut Anforderung, Begründung.
Schreibe noch keine Tests. Ich wähle aus.
```

- Die Trennung **Ideen, Auswahl, Umsetzung** verhindert Tests, die niemand braucht
- **Negativtests** (was soll *nicht* gehen?) bringen oft den größten Gewinn
- Die Liste ist ein Vorschlag: Vergleichen Sie sie mit Ihrer eigenen Vermutung

# Ausgewählte Fälle als Test

```python
import math

def seiten(n: int, pro_seite: int) -> int:
    """Seitenzahl für n Treffer."""
    if pro_seite < 1:
        raise ValueError("pro_seite < 1")
    return math.ceil(n / pro_seite)
```

```python
import pytest
from liste.seiten import seiten

def test_keine_treffer():
    assert seiten(0, 20) == 0

def test_genau_volle_seite():
    assert seiten(40, 20) == 2

def test_ein_treffer_mehr():
    assert seiten(41, 20) == 3

def test_pro_seite_null():
    with pytest.raises(ValueError):
        seiten(10, 0)
```

Vier Fälle aus der Liste, je ein Verhalten pro Test. Die erwarteten Werte `0`, `2`, `3` stehen in der Anforderung, nicht im Code.

# Debug-Prompt mit Hypothesen

Sechs Angaben, dann Hypothesen statt Fix, im Agenten **Ask**:

```
**Problem:** test_fuehrende_null in tests/test_import.py schlägt fehl.
**Erwartet:** Laut #file:docs/import.md bleibt "007311" als Text erhalten.
**Tatsächlich:** lies_kunden liefert "7311".
**Reproduktion:** python -m pytest tests/test_import.py -k fuehrende_null
**Relevanter Code:** #file:kunden/import_csv.py, Testausgabe unten
**Bisher geprüft:** Die CSV-Datei enthält 007311 (mit head geprüft).

Nenne bis zu drei Hypothesen, nach Wahrscheinlichkeit geordnet.
Gib zu jeder eine Prüfung an, die sie widerlegen würde,
und was ich dabei sehen müsste. Ändere keinen Code.
```

- Die Prüfung führen **Sie** aus: ein einzelner Test, `python -c "..."`, ein Blick in die Datei
- Erst wenn eine Hypothese ihre Prüfung übersteht: Wechsel zum **Agent**, minimaler Fix für genau diese Ursache

> **Hypothesen vor Fix.** Ein Fix ohne bestätigte Ursache verschiebt den Fehler oft nur an eine andere Stelle.

# Kontext für Fehler

| Was soll in den Kontext | Referenz im Chat |
|---|---|
| fehlgeschlagener Test | `#execute/testFailure` |
| Fehler und Warnungen aus **Problems** | `#read/problems` |
| letzter Terminalbefehl | `#read/terminalLastCommand` |
| markierter Text im Terminal | `#read/terminalSelection` |
| Datei, Auswahl, Symbol | `#<datei>`, `#selection`, `#<symbol>` |

- `/fix` im Chat: Fix vorschlagen lassen, erst nach bestätigter Hypothese
- **Terminal-Inline-Chat** mit `Ctrl+I` im Terminal: Frage direkt zur Ausgabe
- Die alten Kurzformen `#testFailure`, `#problems`, `#terminalSelection` funktionieren weiter

Quelle: VS Code Docs, Tools and context reference; VS Code 1.107 Release Notes

# Refactoring mit Invarianten

```
Refactoring von gebuehr_cent in #file:konto/gebuehren.py, Ziel: Lesbarkeit.
Invarianten: gleiche Ergebnisse und gleiche Fehler für alle Eingaben,
Signatur bleibt, keine neuen Abhängigkeiten, tests/test_gebuehren.py bleibt grün.
Ein Schritt, ein Diff. Danach python -m pytest tests/test_gebuehren.py ausführen.
```

## Vorher

```python
def gebuehr_cent(kontoart, saldo_cent):
    if kontoart == "giro":
        if saldo_cent < 0:
            return 990
        else:
            return 490
    else:
        if kontoart == "tagesgeld":
            return 0
        else:
            raise ValueError(kontoart)
```

## Nachher

```python
GRUND_CENT = {"giro": 490, "tagesgeld": 0}
UEBERZOGEN_GIRO_CENT = 990

def gebuehr_cent(kontoart, saldo_cent):
    if kontoart not in GRUND_CENT:
        raise ValueError(kontoart)
    if kontoart == "giro" and saldo_cent < 0:
        return UEBERZOGEN_GIRO_CENT
    return GRUND_CENT[kontoart]
```

> Ein Ziel je Auftrag, Invarianten ausdrücklich. Prüfen Sie den Rand: Eine unbekannte Kontoart wirft vorher und nachher denselben `ValueError`.

# Review-Prompt

Erster Durchgang vor dem menschlichen Review, im Agenten **Ask**:

```
#search/changes

Prüfe diese Änderung. Fokus:
- Fehler und Grenzfälle, die die Tests nicht abdecken
- Fehlerpfade: None, leere Eingaben, Exceptions
- Sicherheit: ungeprüfte Eingaben in SQL, Shell oder Logs

Je Befund: Datei und Zeile, Risiko (hoch, mittel, niedrig), Vorschlag.
Ändere keine Dateien.
```

- **`#search/changes`** hängt die aktuellen Änderungen an (alte Form `#changes`)
- VS Code bringt zusätzlich ein eingebautes **Code Review** für markierten Code mit (Review selection)
- Sie entscheiden je Befund: übernehmen, verwerfen, nachfragen

# Docstring und Commit-Nachricht

- **`/doc`** im Inline-Chat (`Ctrl+I`) erzeugt Dokumentationskommentare für den markierten Code
- Vorgabe dazu: Parameter, Rückgabe, Fehlerfälle; kein Kommentar, der den Code nacherzählt
- **Commit-Nachricht:** das Funkeln-Symbol im Eingabefeld der Quellcodeverwaltung erzeugt einen Vorschlag, den Sie vor dem Commit lesen und anpassen

```
git commit -m "fix(import): Kundennummern als Text einlesen" \
  -m "Kundennummern sind Kennungen. int() entfernte führende Nullen,
Kunden mit 00-Präfix waren nicht mehr auffindbar."
```

> Erste Zeile: Typ, Bereich, was. Rumpf: **warum**. Was sich geändert hat, zeigt der Diff.

# Anti-Pattern: Tests grün gepromptet

Auftrag: *„Die Tests sind rot. Bring sie in Ordnung.“* Ein Agent kann darauf so reagieren:

- erwartete Werte an das anpassen, was der Code gerade liefert
- Tests löschen oder mit `skip` markieren
- einen Sonderfall in den Produktionscode einbauen, der nur den Test bedient
- einen hart kodierten Rückgabewert zurückgeben

```
Die Tests beschreiben das erwartete Verhalten laut #file:docs/anforderung.md.
Ändere den Produktionscode, nicht die Tests.
Hältst du einen Test für falsch: nichts ändern, Begründung mit Fundstelle nennen.
```

> Review-Frage bei jedem Diff mit Testdatei: **War der Test falsch oder der Code?** `git show --stat` zeigt, welche Dateien ein Commit geändert hat.

# Zusammenfassung Teil III

- **Ideen vor Umsetzung:** Fälle erfragen, auswählen, dann `/tests` oder Agent
- **Hypothesen vor Fix:** jede mit einer Prüfung, die sie widerlegen könnte; Ask, bis eine bestätigt ist
- **Kontext für Fehler:** `#execute/testFailure`, `#read/problems`, Terminalausgabe
- **Refactoring mit Invarianten:** ein Ziel, ein Schritt, Tests grün
- **Test-Diffs prüfen:** War der Test falsch oder der Code?

TEIL IV

# Copilot anpassen

Regeln, Aufträge und Werkzeuge einmal ablegen, statt sie in jeden Prompt zu tippen.

# Lernziele

## Instruktionen

Repo-weite, pfadbezogene und `AGENTS.md`-Instruktionen schreiben und ihre Wirkung prüfen

## Wiederverwenden

Wiederkehrende Aufträge als Prompt File, Custom Agent oder Skill ablegen

## Werkzeuge

MCP-Server in VS Code anbinden und Werkzeugfreigaben setzen

## Kontext teilen

Copilot Spaces als gebündelten Kontext für Fragen einsetzen

# Anpassungsebenen

ORGANISATION

**Organisation**

nur Business und Enterprise

Org-Instruktionen in den Einstellungen

Org-Agenten: Repo .github oder .github-private, agents/

REPOSITORY

**Repository**

versioniert, gilt für das Team

.github/copilot-instructions.md

.github/instructions/*.instructions.md

AGENTS.md

.github/prompts/*.prompt.md

.github/agents/*.agent.md

.github/skills/<name>/SKILL.md

.vscode/mcp.json

PERSÖNLICH

**Persönlich**

nur für Sie

Instruktionen: Einstellungen auf GitHub.com, ~/.copilot/

~/.copilot/agents/, ~/.copilot/skills/

# .github/copilot-instructions.md

Eine Datei für das ganze Repository, Markdown ohne Frontmatter:

```
# Projektregeln

Python 3, Tests mit pytest. Keine neuen Pakete ohne Rückfrage.

## Code
- Typannotationen an allen öffentlichen Funktionen
- Geldbeträge als int in Cent, nie als float

## Tests
- Fachliche Erwartungen stehen in docs/
- Widersprechen sich Test und Code: erst nachfragen, dann ändern

## Antworten
- Auf Deutsch, Fachbegriffe wie im Code
```

- Anlegen mit `/init` im Chat oder `copilot init` im Terminal
- Kurze, prüfbare Regeln: Sie sollen im Diff erkennen, ob Copilot sie befolgt hat

Quelle: GitHub Docs, Repository instructions; VS Code Docs, Custom instructions

# Pfadbezogene Instruktionen

Datei `.github/instructions/tests.instructions.md`, gilt für die Dateien aus `applyTo`:

```
---
applyTo: "tests/**/*.py"
---
- Ein Test prüft ein Verhalten, der Name sagt welches
- Erwartete Werte als Literal, nie aus dem Produktionscode berechnet
- Gemeinsame Fixtures in tests/conftest.py
```

- **`applyTo`**: Glob-Muster, mehrere durch Komma getrennt, z. B. `"**/*.ts,**/*.tsx"`
- Ablage beliebig tief unter `.github/instructions/`, Endung `.instructions.md`
- Anlegen mit **`/create-instructions`** im Chat

Quelle: GitHub Docs, Repository instructions; VS Code Docs, Custom instructions

# AGENTS.md

Anweisungen für Agenten: Build, Tests, Arbeitsweise. Kein Frontmatter.

```
# AGENTS.md

## Build und Tests
- Abhängigkeiten: pip install -r requirements.txt
- Alle Tests: python -m pytest
- Ein Test: python -m pytest tests/test_import.py -k fuehrende_null

## Arbeitsweise
- Vor jeder Änderung die betroffenen Tests ausführen
- Keine Änderungen unter migrations/
```

- Beliebig viele im Repository; für eine Datei gilt die **nächstgelegene** `AGENTS.md`
- Alternativ liest Copilot eine `CLAUDE.md` oder `GEMINI.md` im Repo-Root

Quelle: GitHub Docs, Repository instructions; GitHub Docs, Custom instructions

# Prompt Files

Wiederkehrender Auftrag als Datei `.github/prompts/review.prompt.md`:

```
---
name: review
description: 'Änderung auf Fehler und fehlende Tests prüfen'
argument-hint: 'Fokus, z. B. Fehlerpfade'
agent: 'ask'
---
Prüfe die Änderungen in ${input:datei:Pfad zur geänderten Datei}.
Fokus: ${input:fokus:Fehlerpfade}.
Regeln für Tests: #file:.github/instructions/tests.instructions.md

Je Befund: Zeile, Risiko (hoch, mittel, niedrig), Vorschlag.
Ändere keine Dateien.
```

- Aufruf im Chat mit **`/review`**; anlegen mit `/create-prompt`
- Frontmatter: `description`, `name`, `argument-hint`, `agent` (`ask`, `agent`, `plan` oder ein Custom Agent), `model`, `tools`
- Im Text: `${input:var}`, `${input:var:placeholder}`, `#file:pfad`, `#tool:<name>`

Quelle: VS Code Docs, Prompt files

# Custom Agents

Eigene Rolle mit begrenzten Werkzeugen: `.github/agents/testplaner.agent.md`

```
---
name: testplaner
description: 'Plant Testfälle aus der Anforderung, ändert keinen Code'
tools: ['search', 'read']
handoffs:
  - label: Tests umsetzen
    agent: agent
    prompt: Setze die ausgewählten Testfälle als pytest-Tests um.
    send: false
---
Du planst Tests. Lies Anforderung und Code. Liste je Testfall
Eingabe, erwartetes Ergebnis und die Fundstelle in der Anforderung.
Schreibe keine Dateien.
```

- **`tools`** begrenzt, was der Agent darf; **`model`** legt das Modell fest; `description` ist für Agenten auf GitHub Pflicht
- **`handoffs`**: Übergabe an einen anderen Agenten mit `label`, `agent`, `prompt`, `send` (Standard `false`)
- Persönlich unter `~/.copilot/agents`; früher **Custom Chat Modes** (`.chatmode.md` in `.github/chatmodes`), alte Dateien laufen weiter

Quelle: VS Code Docs, Custom agent file reference; GitHub Docs, Custom agents configuration

# Agent Skills

Ordner mit Anleitung nach offenem Standard (agentskills.io): `.github/skills/pytest-regeln/SKILL.md`

```
---
name: pytest-regeln
description: 'Regeln für neue pytest-Tests. Verwenden, wenn Tests geschrieben oder geändert werden.'
---
# Neue Tests

1. Anforderung in docs/ lesen, erwartete Werte von dort übernehmen
2. Testdatei tests/test_<modul>.py, ein Verhalten je Test
3. python -m pytest ausführen und die Ausgabe zeigen
```

- **`name`**: Pflicht, Kleinbuchstaben, gleich dem Ordnernamen, höchstens 64 Zeichen; **`description`**: Pflicht, höchstens 1.024 Zeichen
- Aufruf mit `/pytest-regeln`; `user-invocable` und `disable-model-invocation` steuern, ob Sie oder das Modell den Skill starten
- Weitere Orte: `.agents/skills/`, `.claude/skills/`, persönlich `~/.copilot/skills/`

Quelle: VS Code Docs, Agent skills; GitHub Docs, Customization cheat sheet

# MCP-Server in VS Code

Konfiguration im Workspace: `.vscode/mcp.json`, Schlüssel `servers`

```
{
  "servers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/"
    }
  }
}
```

- **GitHub MCP Server:** für alle GitHub-Nutzer, Anmeldung per OAuth oder PAT; alternativ über die Erweiterungsansicht, Suche `@mcp github`, **Install**
- Prüfen mit dem Befehl **MCP: List Servers**
- **Werkzeugfreigabe:** MCP-Tools laufen über dieselben Freigaben wie andere Tools (**Chat: Manage Tool Approval**, gruppiert je Server)
- Geheimnisse über `inputs` mit `"password": true` abfragen, nicht in die Datei schreiben

> Business und Enterprise: Die Richtlinie **MCP servers in Copilot** entscheidet, ob MCP überhaupt läuft. Sie ist **standardmäßig aus.**

Quelle: VS Code Docs, MCP configuration reference; GitHub Docs, Setting up the GitHub MCP Server; GitHub Docs, About MCP

# Copilot Spaces

- **Gebündelter Kontext** für Fragen an Copilot: Repos, Dateien, PRs, Issues, Freitext, Bilder, Uploads
- Anlegen unter `github.com/copilot/spaces` > **Create space: Instructions** plus **Sources**
- **Repo** als Quelle: Copilot sucht darin; **Datei**: voller Inhalt in jedem Kontextfenster
- Für jede Copilot-Lizenz, auch **Copilot Free**; Fragen verbrauchen AI Credits
- Teilen: Org-Spaces mit Rollen; persönliche Spaces öffentlich (nur lesend), für bestimmte Nutzer oder privat
- In der IDE über den GitHub MCP Server, Toolset `copilot_spaces`, nur im **Agent**

Quelle: GitHub Docs, About Copilot Spaces; GitHub Docs, Creating Spaces; GitHub Docs, Spaces in your IDE

# Wo wirkt welche Anpassung

| Funktion | VS Code | Visual Studio | JetBrains | GitHub.com | Copilot CLI |
|---|---|---|---|---|---|
| Custom Instructions | ja | ja | Preview | ja | ja |
| Prompt-Dateien | ja | ja | Preview | nein | nein |
| Custom Agents | ja | ja | Preview | ja | ja |
| Subagents | ja | nein | Preview | nein | ja |
| Agent Skills | ja | ja | Preview | ja | ja |
| Hooks | Preview | nein | nein | ja | ja |
| MCP | ja | ja | ja | ja | ja |

> **Prompt-Dateien** laufen in VS Code nur im **Local-Harness.** Für den Copilot-Harness verweist die Doku auf **Skills.**

Quelle: GitHub Docs, Customization cheat sheet; VS Code Docs, Prompt files

# Welche Instruktionen jede Oberfläche liest

| Oberfläche | gelesene Instruktionen |
|---|---|
| VS Code Inline-Vorschläge | keine |
| VS Code Chat | repo-weit, pfadbezogen (nur für passende Dateien), `AGENTS.md` |
| VS Code Code Review | nur repo-weit |
| Chat auf GitHub.com | persönlich, repo-weit, Organisation |
| Code Review auf GitHub.com | repo-weit, pfadbezogen, `AGENTS.md`, `REVIEW.md`, Organisation (vom Head-Branch) |
| Cloud Agent | repo-weit, pfadbezogen, `AGENTS.md`, Organisation |
| Copilot CLI | repo-weit (Git-Wurzel und Arbeitsordner), `AGENTS.md`, `.github/instructions/`, persönlich |

> `excludeAgent: "code-review"` oder `"cloud-agent"` im Frontmatter nimmt eine pfadbezogene Datei gezielt aus. Verschachtelte `AGENTS.md` liest VS Code erst mit `chat.useNestedAgentsMdFiles`.

Quelle: GitHub Docs, Custom instructions support; VS Code Docs, Custom instructions

# Zusammenfassung Teil IV

- **Instruktionen:** `.github/copilot-instructions.md` für das Repo, `*.instructions.md` mit `applyTo` für Pfade, `AGENTS.md` für Agenten
- **Aufträge wiederverwenden:** Prompt File (`/name`), Custom Agent (Rolle, Werkzeuge, Handoffs), Skill (`SKILL.md`)
- **MCP:** `.vscode/mcp.json`, Freigaben je Werkzeug, in Business und Enterprise eine Richtlinie, die standardmäßig aus ist
- **Spaces:** Kontext bündeln und teilen, auch mit Copilot Free
- **Wirkung prüfen:** Jede Oberfläche liest nur einen Teil der Dateien

TEIL V

# Copilot außerhalb der IDE

Copilot arbeitet auch im Terminal und direkt auf GitHub.com, jeweils mit eigenen Regeln.

# Lernziele

## Copilot CLI

Sitzungen starten, **Modi** wählen, **Werkzeuge** freigeben und sperren

## GitHub.com

**Chat**, **PR-Zusammenfassung** und **Code Review** gezielt einsetzen

## Cloud Agent

Den Weg vom **Issue zum Entwurfs-PR** und seine **Schutzregeln** kennen

# Copilot CLI installieren

```
npm install -g @github/copilot          # alle Plattformen, Node.js 22 oder neuer
winget install GitHub.Copilot           # Windows
brew install --cask copilot-cli         # macOS und Linux
curl -fsSL https://gh.io/copilot-install | bash   # macOS und Linux
```

```
cd mein-projekt
copilot                                 # Start; beim ersten Mal in der Sitzung: /login
copilot update                          # CLI aktualisieren
```

Quelle: GitHub Docs, Installing Copilot CLI

# Interaktiv oder programmatisch

| Aufruf | Wirkung |
|---|---|
| `copilot` | interaktive Sitzung |
| `copilot -i "PROMPT"` | interaktive Sitzung, die den Prompt sofort ausführt |
| `copilot -p "PROMPT"` | führt den Prompt aus und beendet sich |
| `copilot -sp "PROMPT"` | wie `-p`, gibt nur die Antwort aus |

> `-i` ist **kein** Einmal-Prompt: Die Sitzung bleibt offen. Ausführen und beenden heißt `-p` (lang `--prompt`).

> Mit `-p` läuft kein Dialog. Was der Lauf ändern darf, geben Sie beim Aufruf frei, zum Beispiel mit `--allow-tool`.

Quelle: GitHub Docs, CLI command reference; CLI quickstart

# Modi im CLI und in der Copilot App

- **Standard** (interaktiv): Copilot fragt vor Schreibzugriffen, riskanten Shell-Befehlen und URLs nach
- **Plan**: Copilot erstellt zuerst einen Plan; Start mit `--plan` oder `/plan`
- **Autopilot**: Copilot arbeitet selbstständig in mehreren Schritten weiter; Start mit `--autopilot` oder `/autopilot ZIEL`, begrenzen mit `--max-autopilot-continues=N`
- Wechsel während der Sitzung mit `Shift+Tab`, beim Start mit `--mode=plan`

> **Copilot App**: Desktop-App auf Basis des CLI, für **alle Pläne**, mit denselben drei Modi. Jede Sitzung läuft in eigenem Git-Worktree und Branch. Aus dem CLI starten Sie sie mit `copilot app`.

# Werkzeugfreigaben

Standard: Lesen und Suchen laufen automatisch. Schreiben, riskante Shell-Befehle und URLs fragen nach.

```
# alle git-Befehle erlauben, git push sperren
copilot --allow-tool='shell(git:*)' --deny-tool='shell(git push)'

# Einmal-Prompt mit genau einer Freigabe
copilot -p "Fasse die letzten fünf Commits zusammen" --allow-tool='shell(git)'

# ein einzelnes MCP-Tool oder einen ganzen MCP-Server freigeben
copilot --allow-tool='MyMCP(create_issue)'
copilot --allow-tool='MyMCP'
```

- **Arten:** `shell(...)`, `write(...)`, `read(...)`, `url(...)`, `memory`, `SERVERNAME(tool)`
- **Deny schlägt Allow**, auch bei `--allow-all` und bei gespeicherten Freigaben
- `--allow-all` (oder `--yolo`) gibt Tools, Pfade und URLs frei: nur in Sandbox, VM oder Container

Quelle: GitHub Docs, Allowing and denying tool use

# Arbeitsordner: Vertrauen und Instruktionen

- Beim Start fragt das CLI, ob Sie dem Ordner vertrauen: **Yes**, **Yes, and remember this folder**, **No**
- Gemerkte Ordner stehen in `~/.copilot/config.json` unter `trustedFolders`; weitere Ordner mit `/add-dir PFAD` oder `--add-dir`, Übersicht mit `/list-dirs`
- Nicht im **Home-Verzeichnis** starten und nicht in Ordnern mit unbekannten ausführbaren Dateien oder sensiblen Daten

> Die Begrenzung auf vertraute Ordner ist laut GitHub eine **Heuristik**. Außerhalb davon garantiert GitHub keinen Schutz.

> Das CLI liest Instruktionen aus Repo und Home-Verzeichnis und führt sie zusammen: `.github/copilot-instructions.md` (Git-Wurzel und Arbeitsordner), `.github/instructions/`, `AGENTS.md` und persönlich `$HOME/.copilot/copilot-instructions.md`. **Prompt Files** liest es nicht.

Quelle: GitHub Docs, Using Copilot CLI; CLI command reference; Customization cheat sheet

# Slash-Befehle im CLI

## Sitzung

`/login`, `/model`, `/context`, `/compact`, `/clear`, `/resume`, `/usage`

## Arbeit

`/plan`, `/autopilot`, `/agent`, `/init`, `/review`, `/diff`, `/delegate`

## Kontrolle

`/permissions`, `/reset-allowed-tools`, `/add-dir`, `/list-dirs`, `/mcp`, `/undo`

> Die aktuelle Referenz kennt **kein** `/explain`, `/suggest` oder `/revise`. Ältere Lernmodule nennen sie noch. Code erklären lassen Sie sich mit einem normalen Prompt.

Quelle: GitHub Docs, CLI command reference

# Copilot auf GitHub.com

- **Chat** unter `github.com/copilot` und auf jeder Seite; Modellwahl unten im Chat, Bilder und PDF als Anhang
- Kontext per `@`: Issues, Pull Requests, Dateien, Repositories, Discussions
- **PR-Zusammenfassung**: Beschreibungsfeld > **Copilot actions** > **Summary**; leer beginnen, vorhandener Text wird ignoriert
- Grenzen: Dateien mit mehr als 400 geänderten Zeilen fließen nicht ein, die Zusammenfassung aktualisiert sich nicht selbst, nicht in Copilot Free

> **Prüfungshinweis Spark**: Der Studienführer nennt GitHub Spark noch. Spark nimmt seit 04.08.2026 keine neuen Nutzer an; Zugriff zum Export gab es bis 31.08.2026.

Quelle: GitHub Docs, Chat in GitHub; PR summary; Changelog vom 04.08.2026

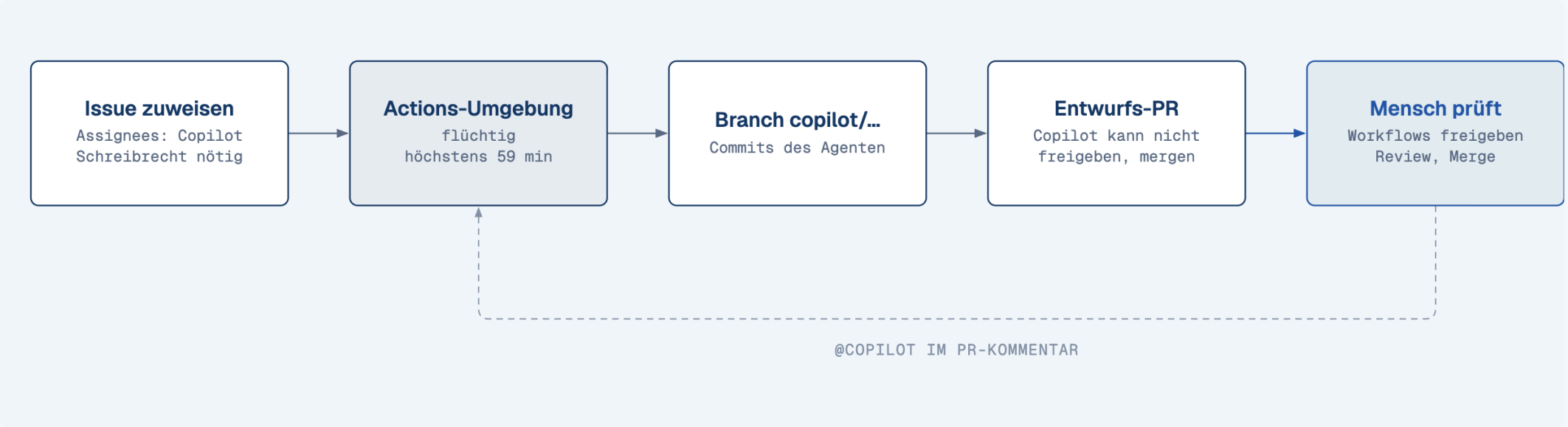
# Copilot Code Review

- **Anfordern:** PR > **Reviewers** > **Copilot** > **Request**; dauert meist unter 30 Sekunden
- Copilot gibt ein Review der Art **Comment** ab: kein Approve, zählt standardmäßig nicht zu den Pflicht-Freigaben
- **Automatisch:** persönlich unter Copilot settings > **Code review**; für Repo und Org per Branch-Ruleset **Automatically request Copilot code review**
- **Review-Instruktionen:** `.github/copilot-instructions.md`, `.github/instructions/**/*.instructions.md`, `AGENTS.md`, `REVIEW.md`, gelesen vom **Head-Branch**
- **Fix with Copilot** übergibt einen Kommentar an den Cloud Agent

Quelle: GitHub Docs, Using code review; Configuring code review

# Cloud Agent: vom Issue zum Entwurfs-PR

Sie weisen ein Issue **Copilot** zu; der **Cloud Agent** (früher **coding agent**) arbeitet allein und liefert einen Entwurfs-PR.



# Cloud Agent: Schutzregeln

- Auslösen dürfen nur Nutzer mit **Schreibrecht**
- Push nur auf einen eigenen Branch mit Präfix **`copilot/`** (bei `@copilot` auf den PR-Branch), kein direktes `git push`
- Eine Sitzung läuft höchstens **59 Minuten** in einer flüchtigen Umgebung auf GitHub Actions
- Ergebnis ist ein **Entwurfs-PR:** Copilot kann ihn nicht auf "Ready for review" setzen, nicht freigeben, nicht mergen
- **Wer die Aufgabe gestellt hat, kann den PR nicht freigeben**
- Actions-Workflows laufen erst nach **Approve and run workflows** durch einen Nutzer mit Schreibrecht

Quelle: GitHub Docs, Risks and mitigations (cloud agent); Review Copilot output

# Cloud Agent: Umgebung und Firewall

`.github/workflows/copilot-setup-steps.yml` bereitet die Umgebung vor: genau ein Job `copilot-setup-steps`, wirksam nur vom Default-Branch.

```yaml
name: "Copilot Setup Steps"
on:
  workflow_dispatch:
  push:
    paths: [.github/workflows/copilot-setup-steps.yml]
  pull_request:
    paths: [.github/workflows/copilot-setup-steps.yml]
jobs:
  copilot-setup-steps:
    runs-on: ubuntu-latest
    permissions:
      contents: read
    steps:
      - name: Checkout code
        uses: actions/checkout@v6
```

> Die **Firewall** ist standardmäßig an, mit empfohlener Allowlist. Sie gilt nur für Befehle über das Bash-Tool des Agenten, **nicht** für MCP-Server und Setup-Steps.

Quelle: GitHub Docs, Customize the agent environment; Customize the firewall

# Recap Teil V

- `copilot -p` führt einen Prompt aus und endet; `-i` lässt die Sitzung offen
- **Werkzeugfreigaben** legen fest, was der Agent ohne Rückfrage darf; Deny schlägt Allow
- Der **Arbeitsordner** bestimmt, was das CLI liest und darf
- **Code Review** kommentiert, freigeben müssen Menschen
- Der **Cloud Agent** liefert einen Entwurfs-PR; freigeben kann ihn nur jemand anderes als der Auftraggeber

TEIL VI

# Verwaltung, Datenschutz und Prüfung

Was eine Organisation an Copilot einstellt, was mit Ihren Daten geschieht und wie die Prüfung GH-300 abläuft.

# Lernziele

## Verwaltung

**Richtlinien**, **Content Exclusion**, Audit Log und REST API einordnen

## Daten und Sicherheit

Training und Aufbewahrung kennen, **Prompt Injection** erkennen und begrenzen

## Prüfung

Bereiche, Format und **alte Begriffe** in Fragen kennen

# Business und Enterprise

## Plätze

Org **Settings > Copilot > Access:** für alle oder ausgewählte Mitglieder; Abrechnung ab Zuweisung, nicht ab erster Nutzung

## Richtlinien

Die Enterprise entscheidet zuerst; Organisationen können ihre Vorgaben nicht überschreiben

## Modelle

Org- und Enterprise-Owner **schalten Modelle frei**; eigene API-Schlüssel (BYOK) auf Org-Ebene

## Org-Instruktionen

In den Org-Einstellungen; gelesen von Chat auf GitHub.com, Cloud Agent und Code Review

> Nur in Business und Enterprise: **Org-Richtlinien**, **Content Exclusion**, und GitHub trainiert **nicht** mit Kundendaten. Enterprise hat 3.900 statt 1.900 AI Credits je Nutzer im Pool.

# Richtlinien: Vorrang und Reichweite

- Ort: Enterprise-Reiter **AI controls**, Org **Settings > Copilot > Policies** und **Models**
- Die Enterprise wählt je Richtlinie: aktivieren, deaktivieren oder **Let organizations decide**
- Lizenz über mehrere Orgs derselben Enterprise: meist gilt die **am wenigsten restriktive** Einstellung

| Richtlinie | IDEs | Cloud Agent | CLI | Chat GitHub.com | Code Review |
|---|---|---|---|---|---|
| Suggestions matching public code | ja | nur Annotate | nein | nein | nein |
| Content exclusion | ja | ja | ja | ja | ja |
| MCP servers in Copilot | ja | ja | ja | nein | ja |

**MCP servers in Copilot** ist standardmäßig **aus.** Solange sie aus ist, gibt es in IDE, CLI, App, Cloud Agent und Code Review keine MCP-Server; im Chat auf GitHub.com greift sie nicht.

Quelle: GitHub Docs, Copilot policies; Supported surfaces for policies; About MCP

# Content Exclusion

```
# Repo: Settings > Copilot > Content exclusion
- "/src/some-dir/kernel.rs"
- "secrets.json"
- "secret*"
- "*.cfg"
- "/scripts/**"
```

```
# Org: Schlüssel "*" gilt für alle Repositories
"*": ["**/.env"]
```

- Wirkung: keine Inline-Vorschläge in der Datei, ihr Inhalt fließt nicht in Antworten, Code Review prüft sie nicht
- Änderungen erreichen geladene IDEs erst nach bis zu **30 Minuten**
- Greift **nicht** im **Agent mode** der IDEs, bei Drittanbieter-Agenten, für Symlinks, Remote-Dateisysteme und indirekte IDE-Infos wie Typen und Hover

**CLI offen:** Mehrere Doku-Seiten sagen, das CLI beachtet Content Exclusion; die Anleitung zum Einrichten sagt das Gegenteil. Verlassen Sie sich im CLI nicht darauf.

Quelle: GitHub Docs, Content exclusion; Excluding content

# Filter für öffentlichen Code und Code Referencing

- Einstellung **Suggestions matching public code: Allow** oder **Block**
- **Block** prüft den Vorschlag mit rund 150 Zeichen Umgebung gegen öffentlichen Code auf GitHub; Treffer und Beinahe-Treffer erscheinen nicht
- **Allow:** Copilot zeigt **Code-Referenzen** mit Lizenz und URL, in VS Code unter **View > Output > GitHub Copilot Log (Code References)**
- Mit einem Org-Platz gilt die Einstellung der Organisation

> Den vertraglichen IP-Schutz (Indemnity) für Business und Enterprise gibt es laut Lernmodul nur mit der Org-Einstellung **Block**.

Quelle: GitHub Docs, Find matching code; Microsoft Learn, Contractual protections

# Audit Log und REST API

- Audit Log der Enterprise: **Settings > Audit log**, Suche mit `action:copilot`, Agentenaktivität mit `actor:Copilot`
- Aufbewahrung **180 Tage**; Streaming in ein SIEM empfohlen
- **Nicht enthalten:** lokale Sitzungsdaten wie Prompts

| Zweck | Endpunkt |
|---|---|
| Plätze und Einstellungen der Org | `GET /orgs/{org}/copilot/billing` |
| Alle Platzzuweisungen | `GET /orgs/{org}/copilot/billing/seats` |
| Nutzer hinzufügen, entfernen | `POST`, `DELETE /orgs/{org}/copilot/billing/selected_users` |
| Teams hinzufügen, entfernen | `POST`, `DELETE /orgs/{org}/copilot/billing/selected_teams` |
| Platz eines Nutzers | `GET /orgs/{org}/members/{username}/copilot` |

Quelle: GitHub Docs, Reviewing audit logs; REST API Copilot user management

# Datenschutz: Training und Aufbewahrung

- **Business und Enterprise:** kein Training mit Kundendaten; die Einstellung wird dort gar nicht angezeigt
- **Free, Pro, Pro+, Max:** Seit **24.04.2026** darf GitHub Eingaben, Ausgaben, Code-Schnipsel und Kontext zum Training nutzen
- Abschalten: Copilot settings > **Allow GitHub to use my data for AI model training > Disabled**

| Oberfläche | Aufbewahrung |
|---|---|
| Chat auf GitHub.com | bis 100 Unterhaltungen, Nachrichten 28 Tage |
| IDE-Vorschläge | Prompt wird nach der Antwort verworfen (Lernmodul) |
| CLI und App | volle Sitzung lokal in `~/.copilot/session-state/`, standardmäßig mit dem Konto synchronisiert |
| Cloud Agent | Umgebung wird zerstört, das Log bleibt auf GitHub.com |

Quelle: GitHub Docs, Policies as individual subscriber; Session data

# Was in Prompts gehört

## In der Regel unbedenklich

- öffentlicher oder Open-Source-Code
- allgemeine Fragen, Lernmaterial
- anonymisierte Strukturen und Schemata
- eigener Code, wenn Richtlinie und Vertrag es erlauben

## Nie

- **Geheimnisse**: API-Schlüssel, Passwörter, Zertifikate, Tokens
- **personenbezogene Daten**, Gesundheits- oder Finanzdaten
- **proprietärer Code** ohne ausdrückliche Freigabe
- Inhalte unter **NDA** oder fremder Lizenz

> Content Exclusion hält Dateien aus dem Kontext, aber nicht im Agent mode. Geheimnisse gehören deshalb gar nicht ins Repository, auch nicht in `.env` unter Versionskontrolle.

# Prompt Injection: Muster und Gegenmittel

**Prompt Injection**: Anweisungen, die in Inhalten stecken, die der Agent liest.

## Muster

- "Ignoriere alle bisherigen Anweisungen" in README, Issue oder Code-Kommentar
- versteckter Text: HTML-Kommentare, unsichtbare Zeichen, Base64
- Anweisungen in Werkzeugausgaben und Webseiten
- Aufträge, die nicht zur Aufgabe passen: `curl` an eine fremde URL, Installation aus fremder Paketquelle, Dateien außerhalb des Ordners lesen

## Gegenmittel

- fremden Text im Prompt als Zitat kennzeichnen und ausdrücklich **Daten** nennen
- im CLI nur nötige Werkzeuge freigeben, riskante sperren (`--deny-tool='shell(pip:*)'`), kein `--allow-all`
- Verbote passen auf den aufgerufenen Befehl: `bash skript.sh` ist kein `curl`-Aufruf, auch wenn im Skript `curl` steht
- eingebaut: Cloud Agent filtert versteckte Zeichen und HTML-Kommentare; VS Code gibt URLs in zwei Schritten frei

> Fremde Inhalte (Issue-Text, Webseite, Werkzeugausgabe) sind **Daten, keine Anweisungen.** Werkzeugfreigaben begrenzen den Schaden.

Quelle: OWASP Top 10 for LLM Applications 2025, LLM01 Prompt Injection; GitHub Docs, Allowing and denying tool use; Risks and mitigations (cloud agent); VS Code Docs, Approvals and permissions

# Vertrauen, aber prüfen

Behandeln Sie Copilot-Ergebnisse wie einen Pull Request einer fremden Person.

## Prüfen

- Tests laufen und prüfen echtes Verhalten; geänderte Tests zuerst lesen
- Linter und statische Analyse
- neue Abhängigkeit: Pflege, Lizenz, Herkunft, Begründung im Commit
- Secret-Scan vor dem Push
- Diff Zeile für Zeile lesen, nicht nur die Zusammenfassung

## Unsichere Voreinstellungen

- `innerHTML` statt `textContent` (XSS)
- SQL aus Strings statt Prepared Statements
- `eval()` oder `exec()` für Konfiguration
- `md5` oder `sha1` als Passwort-Hash
- `Access-Control-Allow-Origin: *` in Produktion

# Prüfung GH-300

| Bereich | Gewicht |
|---|---|
| Copilot verantwortungsvoll nutzen | 15–20 % |
| **Copilot-Funktionen nutzen** | **25–30 %** |
| Daten und Architektur | 10–15 % |
| Prompt Engineering und Kontext | 10–15 % |
| Produktivität für Entwickler | 10–15 % |
| Datenschutz, Content Exclusion, öffentlicher Code | 10–15 % |

- **100 Minuten**, beaufsichtigt, interaktive Aufgaben möglich
- bestanden ab **700** Punkten
- Englisch, Spanisch, Portugiesisch (Brasilien), Koreanisch, Japanisch; sonst auf Antrag **30 Minuten mehr**
- Anmeldung über **Pearson VUE**, Wiederholung frühestens nach 24 Stunden
- Zertifikat gilt **2 Jahre**
- kostenloses **Practice Assessment** auf der Zertifizierungsseite

Quelle: Microsoft Learn, GitHub Copilot certification; Study guide GH-300, Fertigkeiten ab 07.08.2026

# Alte Begriffe in Prüfungsfragen

Fragen und Lernmodule nutzen teils Namen, die es so nicht mehr gibt.

| In der Frage | Heute |
|---|---|
| Copilot coding agent | Copilot cloud agent |
| Custom chat modes (`.chatmode.md`) | Custom agents (`.agent.md`) |
| Premium requests (PRU) | GitHub AI Credits |
| Edit mode, Copilot Edits | abgekündigt, Agent empfohlen |
| Copilot knowledge bases | Copilot Spaces |
| Copilot Extensions (GitHub Apps) | MCP-Server |
| `copilot -i` als Einmal-Prompt, `/explain`, `/suggest` | `copilot -p`, normaler Prompt |

> Der Studienführer nennt **Copilot Edits** und **Spark** weiter als Stoff. Ordnen Sie den alten Namen der heutigen Funktion zu.

# Zusammenfassung

## Arbeiten mit Copilot

- Copilot sieht nur, was im **Kontext** ist: offene Dateien und #-Referenzen
- Ein **Plan** deckt Missverständnisse billiger auf als ein Diff
- **Hypothesen vor Fix**; der Test beschreibt das Soll und wird nicht passend gemacht
- **Instruktionen** wirken nur, wo sie gelten; prüfen Sie ihre Wirkung

## Außerhalb der IDE und in der Organisation

- Fremde Inhalte sind **Daten, keine Anweisungen**; Werkzeugfreigaben begrenzen den Schaden
- Cloud Agent und Code Review liefern Vorschläge; die **Freigabe bleibt beim Menschen**
- Der **Plan** bestimmt den Funktionsumfang; Richtlinien und Content Exclusion setzt die Organisation

# Nächste Schritte nach dem Kurs

1. `.github/copilot-instructions.md` für ein echtes Repository anlegen, `/init` als Start
2. Ein **Prompt File** oder einen **Custom Agent** für eine wiederkehrende Aufgabe schreiben
3. Im CLI eine feste Freigabeliste für Ihr Projekt festlegen, ohne `--allow-all`
4. Die **Trainingseinstellung** im eigenen Konto prüfen
5. **Copilot Code Review** auf eigene PRs anfordern und mit Ihrem eigenen Review vergleichen
6. **Practice Assessment** machen, dann die Prüfung bei Pearson VUE buchen

# Anhang

# Quellen


- **GitHub Docs**: Copilot (Pläne, Anpassung, Copilot CLI, Code Review, Cloud Agent, Verwaltung, Content Exclusion), docs.github.com/copilot
- **VS Code Docs**: Copilot in VS Code (Chat, Agenten, Kontext, Custom Instructions, Prompt Files, Custom Agents, MCP), code.visualstudio.com/docs/agents/overview
- **GitHub Changelog**: Umbenennungen und Abrechnung (AI Credits seit 01.06.2026), github.blog/changelog
- **Microsoft Learn**: Kurs GH-300T00 und Lernpfade GitHub Copilot Fundamentals Teil 1 und 2, learn.microsoft.com/training/courses/gh-300t00
- **Microsoft Learn**: Studienführer GH-300 (Stand 07.08.2026) und Zertifizierung GitHub Copilot, learn.microsoft.com/credentials
- **OWASP**: Top 10 for LLM Applications 2025, LLM01 Prompt Injection, genai.owasp.org